



Práctica 4. Procesamiento de Big Data con DataFrames y Clúster

Objetivo

Comprender la diferencia entre el manejo de datos en un dataframe local (limitado por memoria de una sola máquina) y el procesamiento distribuido en un clúster usando datasets grandes.

¿Qué es un Dataset?

Un dataset es un conjunto de datos organizados que pueden estar en archivos CSV, Excel, SQL, JSON, entre otros.

¿Qué es un DataFrame?

Un DataFrame es una estructura tabular (filas y columnas) usada en librerías como pandas (Python) o R, ideal para trabajar con datasets pequeños o medianos en memoria.

Limitaciones de un DataFrame

- Está limitado a la memoria RAM disponible en la máquina.
- Funciona bien con datasets de MBs o pocos GBs, pero no con volúmenes de datos de nivel Big Data.

¿Qué es un Clúster?

Un clúster es un conjunto de computadoras o servidores que trabajan juntos como si fueran una sola unidad para aumentar el rendimiento, la disponibilidad y la escalabilidad de un sistema. Los clústeres se utilizan en aplicaciones que requieren alta disponibilidad, procesamiento paralelo o redundancia de datos.

Características de un Clúster

- Distribución de carga: Divide el trabajo entre varios nodos para mejorar el rendimiento
- Alta Disponibilidad: Si un nodo falla, otro puede tomar su lugar
- Escalabilidad: Se pueden agregar más nodos para aumentar la capacidad
- Tolerancia a fallos: Algunos clústeres están diseñados para seguir funcionando incluso si algunos nodos fallan
- Coordinación: En un clúster, hay un mecanismo de comunicación entre los nodos para organizar tareas.

Componentes de un Clúster

Un clúster generalmente tiene dos tipos de nodos. Master Node (Nodo Maestro) que coordina y administra el clúster. Worker Node (Nodos Trabajadores) que ejecutan tareas asignadas por el nodo maestro. Cada nodo en un clúster tiene su propio hardware, sistema operativo y red, pero están interconectados para trabajar en conjunto.



Tipos de Clústeres

Dependiendo de su propósito, existen varios tipos de clústeres:

Clúster de Alta Disponibilidad (HA)

- Se usa para garantizar que un servicio esté disponible todo el tiempo
- Si un nodo falla, otro nodo toma su lugar automáticamente
- Ejemplo: Servidores web en empresas grandes como Google o Amazon

Clúster de Balanceo de Carga

- Distribuye las solicitudes entre varios servidores para evitar sobrecarga
- Se usa en aplicaciones web, bases de datos y sistemas de almacenamiento
- Ejemplo: Nginx como balanceador de carga para sitios web de alto tráfico

Clúster de Alto Rendimiento (HPC)

- Se usa para cálculos intensivos, como simulaciones científicas o análisis de datos
- Permite procesamiento en paralelo usando múltiples nodos al mismo tiempo

Clúster de Almacenamiento

- Se usa para gestionar grandes volúmenes de datos de forma distribuida
- Ejemplo: Hadoop Distributed File System (HDFS) en Big Data

Optimización de Almacenamiento: CSV vs Parquet

En el ecosistema de Big Data, el motor de procesamiento es solo la mitad del trabajo; la forma en que se guardan los datos es igual de importante. Leer un archivo CSV es costoso en tiempo y procesamiento porque es texto plano. En esta parte, los alumnos:

- Convertirán el dataset original en formato CSV a **Parquet**, un formato de almacenamiento en columnas altamente optimizado para ecosistemas distribuidos.
- Cargarán el nuevo archivo Parquet y compararán el tiempo de lectura con el del CSV original para comprobar la mejora en el rendimiento.



Procesamiento con Spark SQL

Muchos analistas e ingenieros de datos prefieren utilizar la sintaxis tradicional de bases de datos. Spark soporta esto de manera nativa sin perder rendimiento. En esta parte, los alumnos:

- Crearán una vista temporal del DataFrame distribuido.
- Ejecutarán una consulta utilizando la sintaxis estándar de SQL para obtener el agrupamiento por línea del metro.

Por hacer

Vamos a simular un clúster con Kubernetes o Docker en la PC, para ello vamos a crear

1. Procesamiento local (DataFrame)

En esta parte los alumnos:

- **Instalarán Python** en su equipo (si no lo tienen ya).
- **Configurarán el entorno de trabajo** instalando la librería **pandas**.
- **Descargarán un dataset “Afluencia Diaria del Metro (simple)”**
<https://datos.cdmx.gob.mx/dataset/afluencia-diaria-del-metro-cdmx>
- **Comprobarán que el dataset puede cargarse en pandas** y observarán que, al crecer el tamaño de los datos, el rendimiento disminuye.

De acuerdo al siguiente pseudocódigo, puedes tomarlo como guía para realizar la primera parte de la práctica.

1. Cargar dataset

INICIAR cronómetro

LEER archivo CSV de afluencia del metro en un DataFrame

DETENER cronómetro

MOSTRAR tiempo de carga, memoria utilizada y dimensiones

MOSTRAR primeras filas y nombres de columnas

2. Contar registros

INICIAR cronómetro

OBTENER número total de registros

DETENER cronómetro



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



MOSTRAR resultados

3. Filtrar por condición

INICIAR cronómetro

CREAR nuevo DataFrame con registros donde afluencia > 15000

DETENER cronómetro

MOSTRAR cantidad filtrada y top 10 registros con mayor afluencia

4. Calcular estadísticas

INICIAR cronómetro

CALCULAR promedio, máximo, mínimo y suma de la afluencia

DETENER cronómetro

MOSTRAR resultados

5. Agrupar por línea del metro

INICIAR cronómetro

AGRUPAR datos por línea y calcular: cantidad, promedio, suma, máximo, mínimo

DETENER cronómetro

MOSTRAR resultados

6. Agrupar por mes

INICIAR cronómetro

AGRUPAR datos por mes y calcular promedio y total

DETENER cronómetro

MOSTRAR resultados ordenados de Enero a Diciembre

7. Top estaciones con mayor afluencia promedio

INICIAR cronómetro

AGRUPAR por línea y estación, calcular promedio de afluencia

ORDENAR descendente y seleccionar top 10

DETENER cronómetro

MOSTRAR resultados

8. Calcular tiempo total de ejecución



`MOSTRAR tiempo total`

`FIN`

2. Procesamiento distribuido (Clúster con Spark)

En esta parte los alumnos:

- [Instalarán Docker](#)
- [Levantarán un clúster de Spark simulado en su PC](#) con la siguiente configuración mínima:
 - o **1 nodo maestro (spark-master)** encargado de coordinar las tareas.
 - o **2 nodos workers (spark-worker1, spark-worker2)** que ejecutan el procesamiento de los datos.
- [Validarán que el clúster está funcionando](#) entrando a la interfaz web de Spark (<http://localhost:8080>) y verificando que los workers estén activos.
- [Cargarán el mismo dataset en PySpark](#), pero esta vez será distribuido entre los nodos.
- Observarán que, aunque el dataset sea muy grande, el clúster puede manejarlo mejor que pandas porque divide las tareas en paralelo.

INICIO

1. Crear sesión de Spark

`CONECTAR al clúster Spark`

`MOSTRAR versión de Spark`

2. Cargar dataset

`INICIAR cronómetro`

`LEER archivo CSV en un DataFrame distribuido`

`DETENER cronómetro`

`MOSTRAR tiempo de carga`

`MOSTRAR esquema y primeras filas`

3. Contar registros

`INICIAR cronómetro`

`CONTAR número total de filas en el DataFrame`



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



DETENER cronómetro

MOSTRAR resultados

4. Filtrar por condición

INICIAR cronómetro

CREAR DataFrame filtrado donde `afluencia > 15000`

CONTAR registros filtrados

DETENER cronómetro

MOSTRAR resultados y top 10 registros con mayor afluencia

5. Calcular estadísticas generales

INICIAR cronómetro

CALCULAR promedio, máximo, mínimo y suma de la afluencia

DETENER cronómetro

MOSTRAR resultados

6. Agrupar por línea del metro

INICIAR cronómetro

AGRUPAR por línea y calcular: cantidad, promedio, suma, máximo, mínimo

DETENER cronómetro

MOSTRAR tabla de resultados

7. Agrupar por mes

INICIAR cronómetro

AGRUPAR por mes y calcular promedio y total de afluencia

DETENER cronómetro

MOSTRAR resultados

8. Top estaciones con mayor afluencia promedio

INICIAR cronómetro

AGRUPAR por línea y estación, calcular promedio de afluencia

ORDENAR descendente y mostrar top 10

DETENER cronómetro



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



MOSTRAR resultados

9. Análisis por año

INICIAR cronómetro

AGRUPAR por año y calcular promedio, total y cantidad de registros

DETENER cronómetro

MOSTRAR resultados

10. Calcular tiempo total

SUMAR todos los tiempos parciales

MOSTRAR tiempo total de ejecución

11. Cerrar sesión de Spark

FIN

Para poder ejecutar los códigos de manera correcta utilizando Docker, es necesario utilizar un archivo docker-compose.yml:

```
version: '3.8'
services:
  spark-master:
    image: bitnami/spark:latest
    container_name: spark-master
    environment:
      - SPARK_MODE=master
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
      - SPARK_SSL_ENABLED=no
    ports:
      - "8080:8080"
      - "7077:7077"
    volumes:
      - ./data:/opt/bitnami/spark/data

  spark-worker1:
    image: bitnami/spark:latest
    container_name: spark-worker1
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_MEMORY=1G
      - SPARK_WORKER_CORES=1
    depends_on:
```



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



```
- spark-master
volumes:
- ./data:/opt/bitnami/spark/data
spark-worker2:
  image: bitnami/spark:latest
  container_name: spark-worker2
  environment:
    - SPARK_MODE=worker
    - SPARK_MASTER_URL=spark://spark-master:7077
    - SPARK_WORKER_MEMORY=1G
    - SPARK_WORKER_CORES=1
  depends_on:
    - spark-master
  volumes:
    - ./data:/opt/bitnami/spark/data
```

CÓDIGOS

Parte 1:

```
import pandas as pd
import time
import os

print("=== PROCESAMIENTO DE AFLUENCIA DEL METRO CON PANDAS (LOCAL)
===")
print("1. Cargando dataset...")
start_time = time.time()
df = pd.read_csv('afluenciastc_simple_08_2025.csv')

load_time = time.time() - start_time
print(f"Tiempo de carga: {load_time:.2f} segundos")
print(f"Memoria utilizada: {df.memory_usage(deep=True).sum() /
1024**2:.2f} MB")
print(f"Dimensiones del dataset: {df.shape}")

print("\nPrimeras 5 filas del dataset:")
print(df.head())
print(f"\nColumnas disponibles: {list(df.columns)}")

print("\n2. Contando registros...")
start_time = time.time()
total_records = len(df)
count_time = time.time() - start_time
print(f"Total de registros: {total_records:,}")
print(f"Tiempo: {count_time:.2f} segundos")

print("\n3. Filtrando datos...")
start_time = time.time()
filtered_df = df[df['afluencia'] > 15000]
```




INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



```
filter_time = time.time() - start_time
print(f"Estaciones con afluencia > 15,000: {len(filtered_df):,}")
print(f"Tiempo: {filter_time:.2f} segundos")

print("Top 10 registros con mayor afluencia:")
top_afluencia = df.nlargest(10, 'afluencia')[['fecha', 'linea',
'aestacion', 'afluencia']]
print(top_afluencia)

print("\n4. Calculando estadísticas de afluencia...")
start_time = time.time()
avg_afluencia = df['afluencia'].mean()
max_afluencia = df['afluencia'].max()
min_afluencia = df['afluencia'].min()
total_afluencia = df['afluencia'].sum()
avg_time = time.time() - start_time
print(f"Afluencia promedio: {avg_afluencia:,.0f} personas")
print(f"Afluencia máxima: {max_afluencia:,} personas")
print(f"Afluencia mínima: {min_afluencia:,} personas")
print(f"Afluencia total: {total_afluencia:,} personas")
print(f"Tiempo: {avg_time:.2f} segundos")

print("\n5. Agrupando datos por línea...")
start_time = time.time()
grouped_linea = df.groupby('linea').agg({
    'afluencia': ['count', 'mean', 'sum', 'max', 'min']
}).round(0)
grouped_linea.columns = ['Registros', 'Promedio', 'Total', 'Máximo',
'Mínimo']
group_time = time.time() - start_time
print("Estadísticas por línea del metro:")
print(grouped_linea)
print(f"Tiempo: {group_time:.2f} segundos")

print("\n6. Agrupando datos por mes...")
start_time = time.time()
grouped_mes = df.groupby('mes').agg({
    'afluencia': ['mean', 'sum']
}).round(0)
grouped_mes.columns = ['Promedio_Mensual', 'Total_Mensual']
grouped_mes = grouped_mes.reindex(['Enero', 'Febrero', 'Marzo',
'Abril', 'Mayo', 'Junio',
'Julio', 'Agosto', 'Septiembre',
'Octubre', 'Noviembre', 'Diciembre'])
month_time = time.time() - start_time
print("Estadísticas por mes:")
print(grouped_mes)
print(f"Tiempo: {month_time:.2f} segundos")

print("\n7. Top 10 estaciones con mayor afluencia promedio...")
start_time = time.time()
```



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



```
top_estaciones = df.groupby(['linea', 'estacion']).agg({
    'afluencia': 'mean'
}).round(0).sort_values('afluencia', ascending=False).head(10)
top_time = time.time() - start_time
print("Top 10 estaciones con mayor afluencia promedio:")
print(top_estaciones)
print(f"Tiempo: {top_time:.2f} segundos")

total_time = load_time + count_time + filter_time + avg_time +
group_time + month_time + top_time
print(f"\n=== TIEMPO TOTAL")
```



Parte 2:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, count, col, sum as spark_sum, max
as spark_max, min as spark_min
import time

print("1. Conectando al clúster Spark...")
spark = SparkSession.builder \
    .appName("MetroAfluenciaPractica") \
    .master("spark://localhost:7077") \
    .config("spark.executor.instances", "2") \
    .config("spark.executor.memory", "512m") \
    .config("spark.executor.cores", "1") \
    .getOrCreate()

print("Conectado al clúster Spark!")
print(f"Spark Version: {spark.version}")

print("\n2. Cargando dataset en clúster...")
start_time = time.time()

df_spark = spark.read.csv('data/afluenciastc_simple_08_2025.csv',
                          header=True,
                          inferSchema=True)

load_time = time.time() - start_time
print(f"Tiempo de carga: {load_time:.2f} segundos")

print("Esquema del dataset:")
df_spark.printSchema()
print("\nPrimeras 5 filas:")
df_spark.show(5)

print("\n3. Contando registros...")
start_time = time.time()
total_records = df_spark.count()
count_time = time.time() - start_time
print(f"Total de registros: {total_records:,}")
print(f"Tiempo: {count_time:.2f} segundos")

print("\n4. Filtrando datos...")
start_time = time.time()
filtered_df = df_spark.filter(col('afluencia') > 15000)
filtered_count = filtered_df.count()
filter_time = time.time() - start_time
```



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



```
print(f"Estaciones with afluencia > 15,000: {filtered_count:,}")
print(f"Tiempo: {filter_time:.2f} segundos")

print("Top 10 registros con mayor afluencia:")
df_spark.orderBy(col('afluencia').desc()).select('fecha', 'linea',
'estacion', 'afluencia').show(10)

print("\n5. Calculando estadísticas de afluencia...")
start_time = time.time()
stats = df_spark.agg(
    avg('afluencia').alias('promedio'),
    spark_max('afluencia').alias('maximo'),
    spark_min('afluencia').alias('minimo'),
    spark_sum('afluencia').alias('total')
).collect()[0]

avg_time = time.time() - start_time
print(f"Afluencia promedio: {stats['promedio']:,} personas")
print(f"Afluencia máxima: {stats['maximo']:,} personas")
print(f"Afluencia mínima: {stats['minimo']:,} personas")
print(f"Afluencia total: {stats['total']:,} personas")
print(f"Tiempo: {avg_time:.2f} segundos")

print("\n6. Agrupando datos por línea...")
start_time = time.time()
grouped_linea = df_spark.groupBy('linea').agg(
    count('afluencia').alias('registros'),
    avg('afluencia').alias('promedio'),
    spark_sum('afluencia').alias('total'),
    spark_max('afluencia').alias('maximo'),
    spark_min('afluencia').alias('minimo')
).orderBy('linea')

print("Estadísticas por línea del metro:")
grouped_linea.show()
group_time = time.time() - start_time
print(f"Tiempo: {group_time:.2f} segundos")

print("\n7. Agrupando datos por mes...")
start_time = time.time()
grouped_mes = df_spark.groupBy('mes').agg(
    avg('afluencia').alias('promedio_mensual'),
    spark_sum('afluencia').alias('total_mensual')
).orderBy('mes')

print("Estadísticas por mes:")
grouped_mes.show(12)
```



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



```
month_time = time.time() - start_time
print(f"Tiempo: {month_time:.2f} segundos")

print("\n8. Top 10 estaciones con mayor afluencia promedio...")
start_time = time.time()
top_estaciones = df_spark.groupBy('linea', 'estacion').agg(
    avg('afluencia').alias('promedio_afluencia')
).orderBy(col('promedio_afluencia').desc())

print("Top 10 estaciones con mayor afluencia promedio:")
top_estaciones.show(10)
top_time = time.time() - start_time
print(f"Tiempo: {top_time:.2f} segundos")

print("\n9. Analizando tendencia por año...")
start_time = time.time()
yearly_trend = df_spark.groupBy('anio').agg(
    avg('afluencia').alias('promedio_anual'),
    spark_sum('afluencia').alias('total_anual'),
    count('afluencia').alias('registros')
).orderBy('anio')

print("Tendencia anual:")
yearly_trend.show()
year_time = time.time() - start_time
print(f"Tiempo: {year_time:.2f} segundos")

total_time = load_time + count_time + filter_time + avg_time + group_time
+ month_time + top_time + year_time
print(f"\n=== TIEMPO TOTAL: {total_time:.2f} segundos ===")

print("\n=== PARTE 3: OPTIMIZACIÓN CON PARQUET ===")
print("1. Guardando el DataFrame en formato Parquet...")
start_time = time.time()
# Guardamos el archivo usando compresión snappy (estándar en Big Data)
df_spark.write.mode("overwrite").parquet('data/afluencia_metro.parquet')
write_time = time.time() - start_time
print(f"Tiempo de escritura Parquet: {write_time:.2f} segundos")

print("\n2. Leyendo el archivo Parquet...")
start_time = time.time()
df_parquet = spark.read.parquet('data/afluencia_metro.parquet')
# Forzamos una acción para medir el tiempo real de lectura
df_parquet.count()
read_parquet_time = time.time() - start_time
print(f"Tiempo de lectura Parquet: {read_parquet_time:.2f} segundos")
```



```
print(f"Mejora de lectura frente a CSV: {(load_time -
read_parquet_time):.2f} segundos")

print("\n=== PARTE 4: CONSULTAS CON SPARK SQL ===")
print("1. Creando vista temporal...")
# Registramos el DataFrame como una tabla SQL temporal
df_parquet.createOrReplaceTempView("metro_cdmx")

print("2. Ejecutando consulta SQL nativa...")
start_time = time.time()
query = """
    SELECT linea,
           COUNT(*) as registros,
           CAST(AVG(afluencia) AS INT) as promedio,
           SUM(afluencia) as total
    FROM metro_cdmx
    GROUP BY linea
    ORDER BY linea
"""
resultado_sql = spark.sql(query)
resultado_sql.show()
sql_time = time.time() - start_time
print(f"Tiempo de ejecución Spark SQL: {sql_time:.2f} segundos")

spark.stop()
```

Instrucciones Práctica 4: Procesamiento Distribuido del Metro CDMX

Para concluir esta práctica y poner a prueba lo aprendido, realizarás un análisis completo utilizando nuevamente el dataset de Afluencia Diaria del Metro CDMX. Sin embargo, en esta ocasión deberás procesar la información de manera distribuida, aplicando las transformaciones y consultas directamente en tu clúster. Sigue los pasos a continuación para generar tu entregable:

1. Configuración inicial:

- Asegúrate de tener Docker instalado y ejecutándose.
- Levanta tu clúster local de Spark usando el archivo docker-compose.yml (1 Master, 2 Workers).
- Descarga el dataset de Afluencia Diaria del Metro CDMX y colócalo en tu carpeta de volúmenes compartidos (./data).

2. Identificador personal:

- Toma los últimos 4 dígitos de tu número de boleta.
- **Regla de transformación:** Si en esos 4 dígitos aparece el número 0, deberás reemplazarlo por un 1.



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



- *Ejemplo 1:* Boleta: 2020630187 → Últimos 4: 0187 → Identificador: 1187
- *Ejemplo 2:* Boleta: 2019630200 → Últimos 4: 0200 → Identificador: 1211
- Este identificador lo usarás para marcar tus resultados finales.

3. Parte A: Carga y exploración en el Clúster

- Inicializa una SparkSession conectada a tu nodo maestro (spark://localhost:7077).
- Carga el archivo CSV del metro utilizando PySpark.
- Muestra el esquema del DataFrame (printSchema()).
- Muestra los primeros 15 registros.
- Imprime el número total de registros procesados por el clúster.

4. Parte B: Limpieza de datos (Data Quality)

- Revisa la columna estacion. Notarás que hay problemas de codificación (caracteres extraños como "Ã³", "Ã¡" en estaciones como "Pantitlán" o "Zócalo"). Utiliza la función regexp_replace o when para corregir al menos **tres** nombres de estaciones que presenten este problema.
- Verifica que la columna afluencia sea de tipo numérico (Integer). Si está como String, realiza el casteo (cast) correspondiente.
- Filtra el DataFrame para eliminar cualquier registro donde la afluencia sea menor a 0 o nula.

5. Parte C: Transformaciones

- Crea una nueva columna llamada categoria_afluencia que clasifique el flujo de pasajeros de la siguiente manera:
 - "**Baja**" si la afluencia es menor a 10,000.
 - "**Media**" si la afluencia está entre 10,000 y 30,000.
 - "**Alta**" si la afluencia es mayor a 30,000.
- Crea una columna llamada miles_pasajeros que contenga el valor de la afluencia dividido entre 1,000 (redondeado a 2 decimales).

6. Parte D: Análisis con DataFrames vs Spark SQL

- **Usando DataFrames (PySpark nativo):**
 - Calcula el promedio de afluencia agrupado por categoria_afluencia. ¿Qué categoría tiene más registros en total?
 - Obtén el Top 5 de estaciones con mayor afluencia total histórica, pero **solo** de la "Linea 1".
- **Usando Spark SQL:**
 - Crea una vista temporal de tu DataFrame (createOrReplaceTempView("metro")).
 - Escribe una consulta SQL que devuelva la suma total de afluencia por año, ordenada del año más antiguo al más reciente.

7. Parte E: Optimización, Identificador y Resultados



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Big data
Ing. Tania Rodríguez Sarabia



- Agrega una columna llamada `id_estudiante` que contenga tu identificador personal (calculado al inicio) en TODAS las filas del DataFrame final.
- Guarda el DataFrame resultante en formato **Parquet** (no CSV) en una carpeta llamada `resultados_practica4_[TU_ID]`. Utiliza el modo "overwrite".
- Vuelve a leer el archivo Parquet recién guardado, cuenta los registros y muestra el tiempo que tardó esta acción comparado con tu lectura inicial del CSV.

8. Parte F: Spark UI

- Agrega una pausa en tu código (`import time; time.sleep(60)`) antes de ejecutar `spark.stop()`.
- Abre tu navegador en <http://localhost:8080> (Master) y toma una captura de pantalla donde se vean tus *Workers* activos y la aplicación ejecutándose.

9. Conclusión: Responde las siguientes preguntas en tu reporte:

1. ¿Qué diferencias notaste en los tiempos de ejecución al leer el formato Parquet en comparación con el CSV original?
2. ¿Te resultó más sencillo realizar las agregaciones de la Parte D utilizando DataFrames o la sintaxis de Spark SQL? Justifica tu respuesta.
3. ¿Qué ventajas te brinda utilizar un clúster de Spark (Master-Worker) en lugar de Pandas cuando el dataset supera la memoria RAM de tu computadora?
4. Menciona alguna dificultad técnica que hayas enfrentado al levantar los contenedores de Docker y cómo la resolviste.
5. Escribe tu conclusión general con respecto a la práctica.